Real-Time Chat Application using MERN Stack and Socket.IO

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Computer Application (BCA) / Master of Computer Application (MCA)/ Master of Science (Data Science) MSc (DS)

By

Student Name:

Enrollment No: 024MCA110304

Under the guidance of

Guide/Mentor Name: Anurag Goel

Certificate

Copy to the certificate received from Qollabb to be pasted here.

Declaration

I, **Yash Bundhe**, hereby solemnly declare that the project report titled " Real-Time Chat Application using MERN Stack and Socket.IO" submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Computer Application (BCA) / Master of Computer Application (MCA)/ Master of Science (Data Science) MSc (DS)** is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2026-27 under the supervision of Anurag Goel **(Guide/Mentor Name)**.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: Pune
Date:

Student Signature: _____
Student Name: Yash Bundhe
Enrollment No: 024MCA110304

Acknowledgement

I would like to express my sincere gratitude to my guide and mentor, Mr. Anurag Goel, for his valuable guidance, encouragement, and continuous support throughout the development of this project. His insights and suggestions greatly helped me in completing this work successfully.

I also extend my thanks to all the faculty members and the institution for providing the necessary resources and a supportive learning environment. Their guidance played an important role in enhancing my technical knowledge and skills.

Finally, I am deeply grateful to my friends and family for their constant motivation, encouragement, and support during the entire project.

Table of Contents

Abstract

This project presents the development of a Real-Time Chat Application using modern web technologies such as React.js, Node.js, Express.js, MongoDB, and Socket.IO. The main objective of this system is to enable instant communication between users through a secure and scalable platform.

The system allows users to register, log in, create chat rooms, and exchange messages in real-time. The backend is implemented using Node.js and Express.js, while MongoDB is used for storing user and message data. Socket.IO enables real-time bidirectional communication between the client and server.

The system analysis involves identifying user requirements such as authentication, message delivery, and chat room management. The system design includes database schema, API architecture, and UI layout. Implementation focuses on building REST APIs, integrating sockets, and designing responsive interfaces.

Testing includes unit testing and real-time message validation. The application ensures reliability, scalability, and performance. Future scope includes adding AI chatbot integration, voice messaging, and end-to-end encryption.

Chapter 1: Introduction

In today's digital era, communication has become faster and more efficient due to advancements in internet technologies. Real-time communication systems play a crucial role in connecting people across the globe instantly. Chat applications are widely used in both personal and professional environments, enabling users to exchange messages, share information, and collaborate effectively. This project focuses on the development of a **Real-Time Chat Application** using modern web technologies.

The main objective of this project is to design and implement a scalable and efficient chat system that allows users to communicate instantly through a web-based platform. The application provides features such as user authentication, chat room creation, and real-time message exchange. The system is built using the MERN stack, which includes React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for database management. Additionally, Socket.IO is used to enable real-time, bidirectional communication between users.

This project aims to address the need for a simple, customizable, and efficient communication platform. It demonstrates the practical implementation of full-stack development concepts, including RESTful APIs, database integration, and real-time data handling. The system is designed to be user-friendly, secure, and scalable.

1.1 Overview

A real-time chat application is a software system that enables instant communication between users over the internet. With the rapid growth of digital communication, chat applications have become an essential part of everyday life, supporting both personal conversations and professional collaborations. These applications allow users to send and receive messages instantly, create group conversations, and share information efficiently.

This project presents the development of a **Real-Time Chat Application** designed to provide seamless and fast communication between multiple users. The system is built using modern web technologies, including React.js for the user interface, Node.js and Express.js for server-side operations, and MongoDB for data storage. Additionally, Socket.IO is integrated to enable real-time, event-based communication, ensuring that messages are delivered instantly without the need for manual refreshing.

The application allows users to register, log in securely, join chat rooms, and exchange messages in real-time. It also maintains chat history, enabling users to view previous conversations. The system is designed with scalability and performance in mind, ensuring that it can handle multiple users simultaneously.

1.2 Objective

The primary objective of this project is to design and develop a **Real-Time Chat Application** that enables users to communicate instantly over the internet in a secure, reliable, and efficient manner. The system aims to provide a seamless communication platform by integrating modern web technologies and real-time data exchange mechanisms.

The specific objectives of the project are as follows:

- To develop a user-friendly interface that allows users to easily register, log in, and interact with the system.
- To implement secure user authentication and authorization mechanisms to protect user data and ensure privacy.
- To enable real-time messaging between users using event-driven communication technologies such as Socket.IO.
- To design and manage chat rooms where multiple users can join and communicate simultaneously.
- To store and retrieve chat history efficiently using a database system.
- To ensure the system is scalable and capable of handling multiple users at the same time without performance degradation.
- To implement a modular and maintainable architecture using the MERN stack (MongoDB, Express.js, React.js, Node.js).
- To provide a foundation for future enhancements such as AI-based chatbots, multimedia sharing, and advanced security features.

1.3 Scope

The scope of this project is to develop a web-based **Real-Time Chat Application** that enables users to communicate instantly through text messages. The system supports user registration, login, and real-time messaging using modern technologies. It allows users to create or join chat rooms and exchange messages efficiently while maintaining chat history in a database. The application is designed to be scalable, user-friendly, and responsive across devices. This project focuses on core messaging functionality and can be extended in the future to include features such as file sharing, voice/video communication, and AI-based chatbot integration.

1.4 Benefits

The developed **Real-Time Chat Application** offers several advantages in terms of communication, usability, and system efficiency. It provides instant message delivery, allowing users to communicate in real-time without delays. The application ensures secure communication through user authentication, protecting user data and privacy. It is user-friendly and easy to navigate, making it accessible even for non-technical users.

The system is scalable and capable of handling multiple users simultaneously, making it suitable for both personal and professional use. It also maintains chat history, enabling users to access previous conversations when needed. Additionally, the project demonstrates the effective use of modern technologies like the MERN stack and real-time communication tools, helping in skill development and practical learning.

Chapter 2: System Study

2.1 Existing System

Existing chat systems such as **WhatsApp, Facebook Messenger, and Telegram** are widely used for real-time communication. These platforms provide advanced features like instant messaging, multimedia sharing, voice and video calls, and end-to-end encryption. They are highly scalable and support millions of users simultaneously.

However, these systems come with certain limitations from a development and customization perspective. They are closed-source platforms, which restrict developers from modifying or customizing features according to specific requirements. Additionally, their complex architecture and heavy resource usage make them unsuitable for small-scale or academic projects.

For learning and experimental purposes, there is a need for a lightweight, customizable, and easy-to-understand chat system. This project aims to overcome these limitations by developing a simple yet efficient real-time chat application that demonstrates core functionalities such as user authentication, chat rooms, and real-time messaging using modern web technologies.

2.2 Proposed System

The proposed system is a Real-Time Chat Application designed to provide fast, secure, and efficient communication between users over the internet. Unlike existing large-scale chat platforms, this system is lightweight, customizable, and focused on core messaging functionality using modern web technologies.

The application allows users to register and log in securely, create or join chat rooms, and exchange messages in real time. The system uses a full-stack architecture with React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for data storage. Socket.IO is integrated to enable real-time, event-driven communication, ensuring instant message delivery without page refresh.

The proposed system is designed to be user-friendly, scalable, and responsive across different devices. It maintains chat history for future reference and ensures smooth communication between multiple users simultaneously. Additionally, the modular architecture allows easy future enhancements such as file sharing, voice/video communication, and AI chatbot integration.

2.3 Feasibility Study

The feasibility study evaluates whether the proposed **Real-Time Chat Application** can be successfully developed and implemented. It includes technical, economic, and operational feasibility.

- **Technical Feasibility**

The project is technically feasible as it uses widely adopted technologies such as React.js, Node.js, Express.js, MongoDB, and Socket.IO. These technologies are well-documented, open-source, and supported by large developer communities. The system can be developed using standard computing resources without requiring advanced hardware.

- **Economic Feasibility**

The project is cost-effective as it relies on free and open-source tools and frameworks. No expensive licenses or infrastructure are required, making it suitable for academic and small-scale applications. Development and deployment costs are minimal.

- **Operational Feasibility**

The system is easy to use and designed with a user-friendly interface. Users can quickly understand and interact with the application without requiring technical expertise. The system can be easily maintained and updated, ensuring smooth operation.

Chapter 3: System Analysis

3.1 Functional Requirements

Functional requirements define the core features and operations of the **Real-Time Chat Application**:

- User Registration: Users can create a new account by providing required details.
- User Login: Registered users can log in securely using their credentials.
- Authentication: The system validates users before granting access.
- Create/Join Chat Rooms: Users can create new chat rooms or join existing ones.
- Send Messages: Users can send messages in real time.
- Receive Messages: Messages are delivered instantly to all users in the chat room.
- Chat History: The system stores messages for future reference.
- Logout: Users can securely log out of the system.

3.2 Non-Functional Requirements

Non-functional requirements define the quality and performance aspects of the system:

- Performance: The system should deliver messages instantly with minimal delay.
- Scalability: It should support multiple users simultaneously.
- Security: User data must be protected using authentication and secure protocols.
- Reliability: The system should function consistently without failures.
- Usability: The interface should be simple and user-friendly.
- Maintainability: The code should be modular and easy to update.
- Availability: The system should be accessible whenever required.

3.3 User Roles

The system defines the following user roles:

- **User**

- Register and log in to the system
- Join or create chat rooms
- Send and receive messages
- View chat history
- Logout from the system

Chapter 4: System Design

4.1 Introduction

System design is an important phase in the Software Development Life Cycle (SDLC) that transforms the system requirements identified during the analysis phase into a structured and implementable solution. It defines the overall architecture, database structure, workflow, and interactions between different modules of the system.

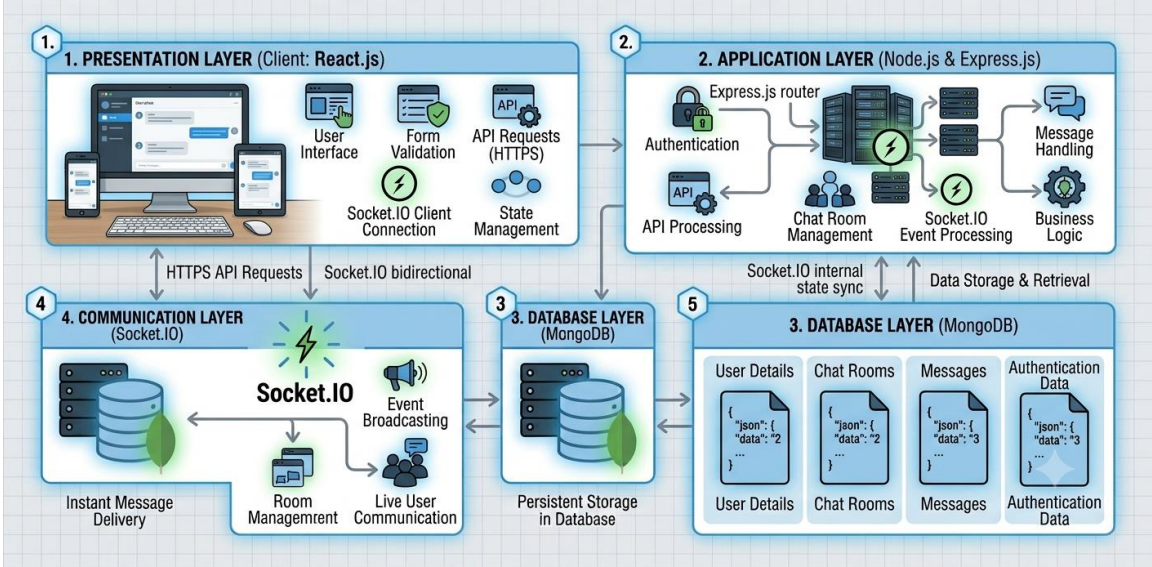
The Real-Time Chat Application is designed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) with Socket.IO for enabling real-time communication. The application follows a client-server architecture, where the frontend communicates with the backend using REST APIs and Socket.IO. The backend processes user requests, interacts with the MongoDB database, and broadcasts messages to connected users.

The system design emphasizes modularity, scalability, maintainability, and performance. Each module

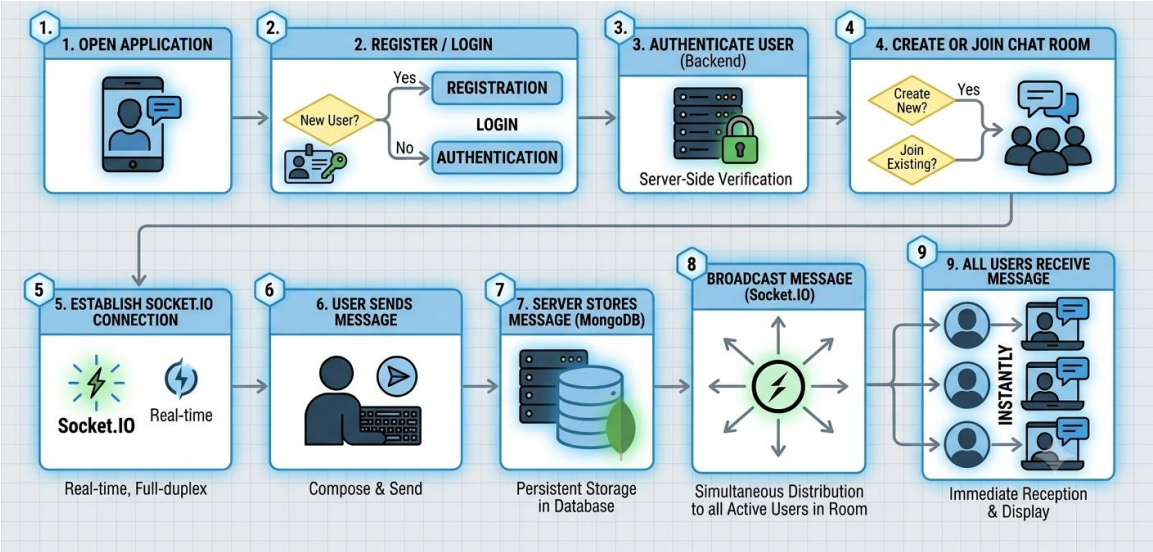
4.2 System Architecture

The Real-Time Chat Application follows a three-tier client-server architecture.

The architecture consists of the following layers:

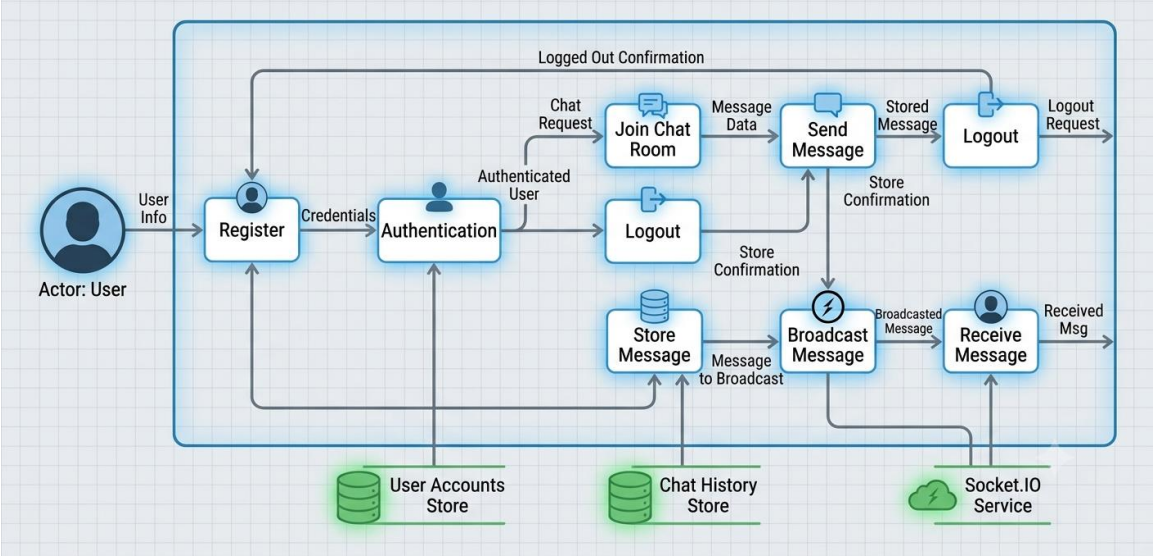


4.3 System Flow



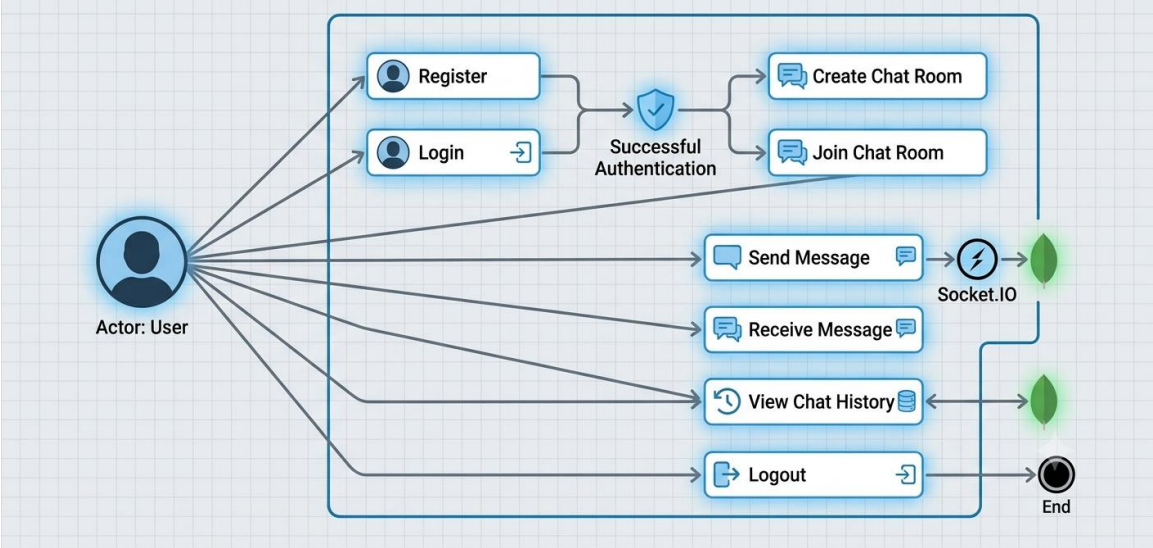
4.4 Data Flow Diagram (DFD)

Level 1 DFD



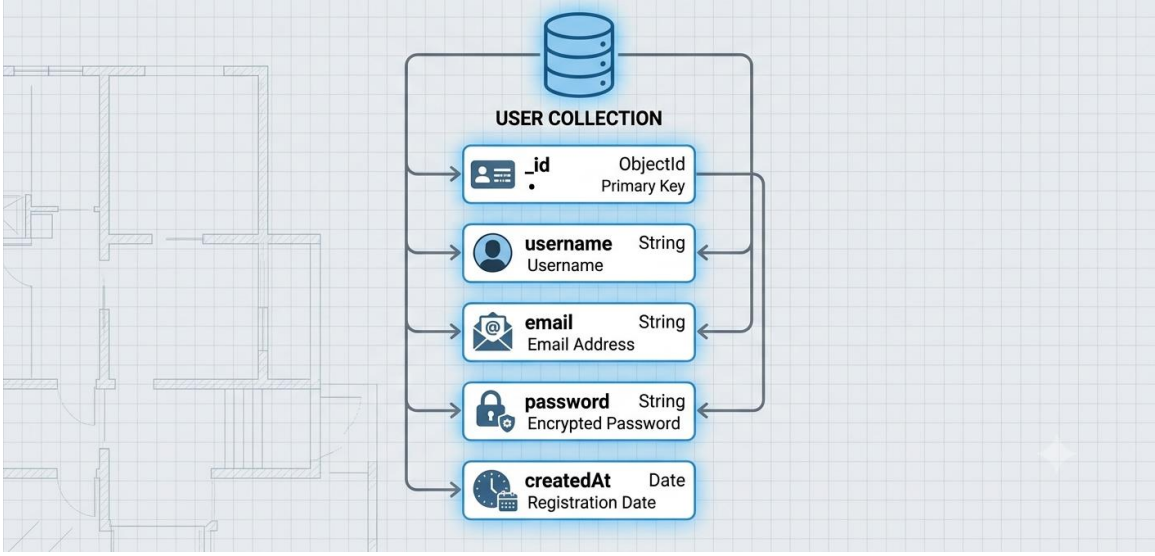
4.5 Use Case Diagram

The Use Case Diagram describes the interactions between users and the system.

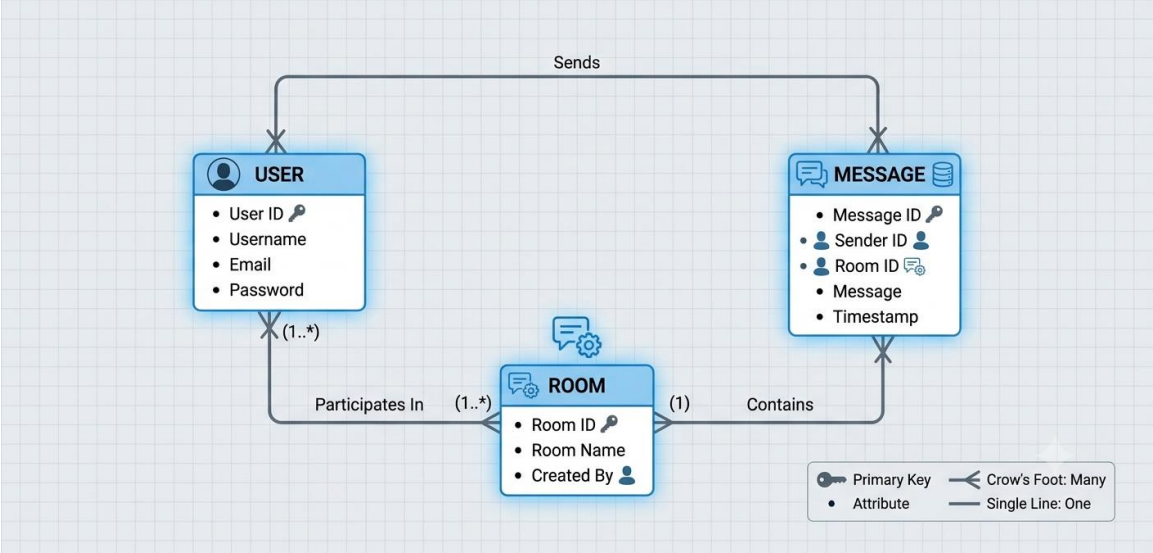


4.6 Database Design

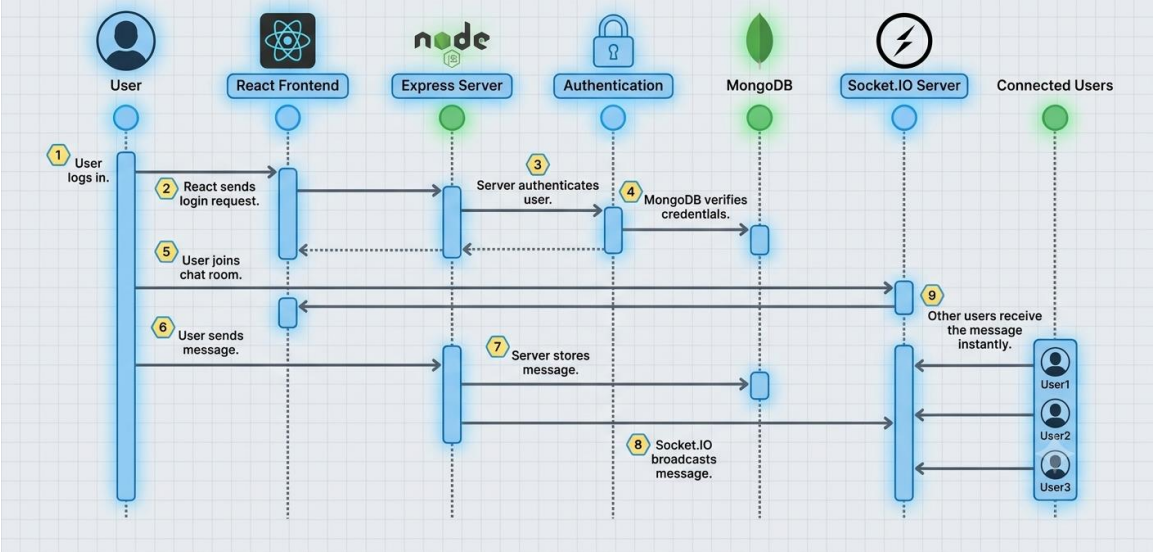
The application uses MongoDB as the database.



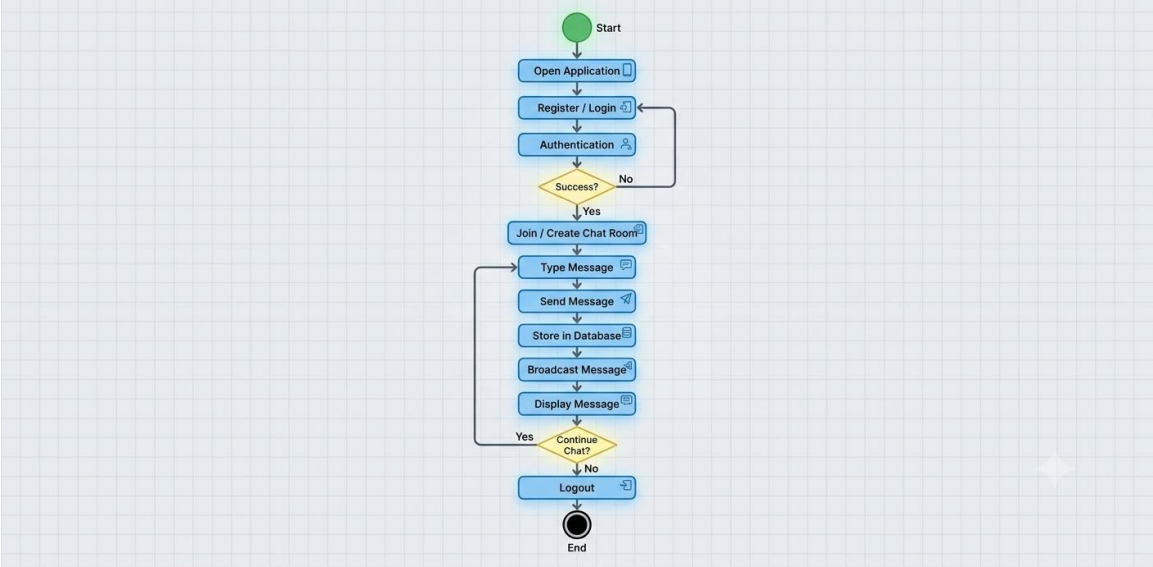
4.7 Entity Relationship (ER) Diagram



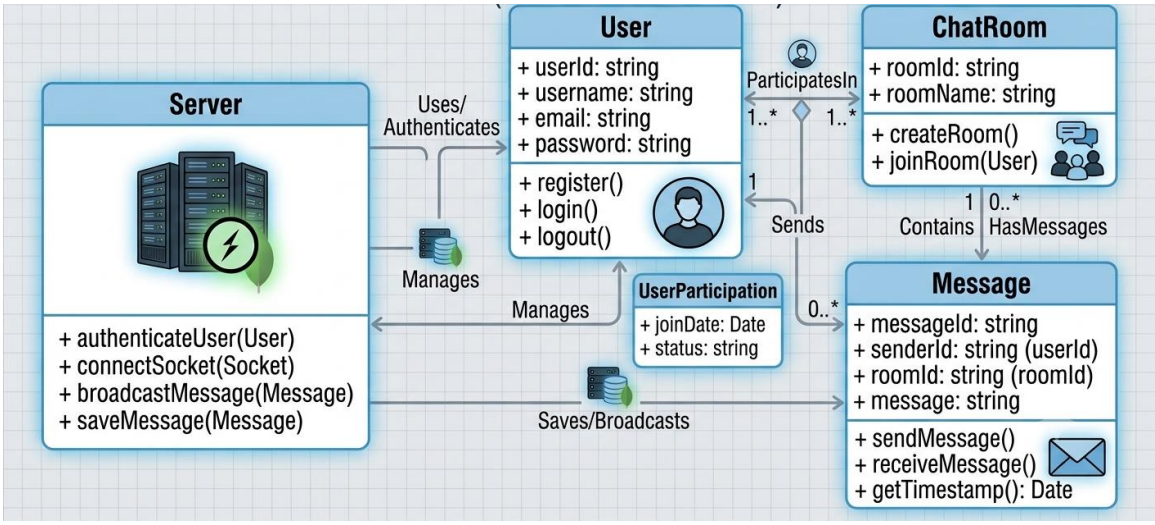
4.8 Sequence Diagram



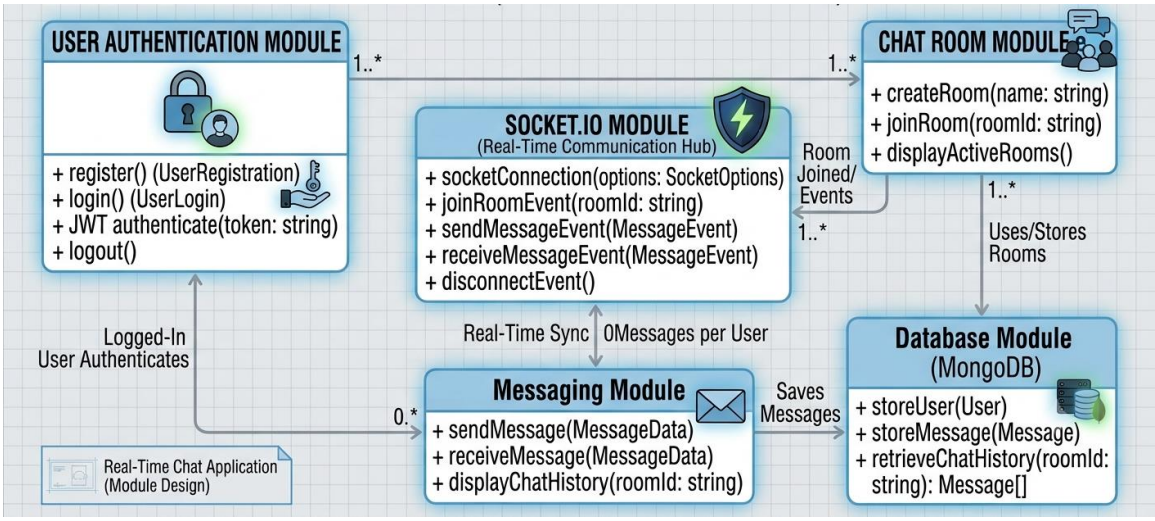
4.9 Activity Diagram



4.10 Class Diagram



4.11 Module Design



4.12 Design Advantages

The proposed system offers several advantages.

- Modular Architecture
- Real-Time Communication
- High Performance
- Secure Authentication
- Scalable Design
- Responsive User Interface
- Easy Maintenance
- Efficient Database Management
- Low Response Time

Easy Future Enhancements

Chapter 5: System Implémentation

This chapter explains how the **Real-Time Chat Application** is developed using modern technologies and how different components of the system are implemented.

5.1 Frontend Implementation (React.js)

The frontend of the application is developed using **React.js**, which provides a dynamic and responsive user interface.

- **Key Features:**
  - Component-based architecture
  - Reusable UI components
  - Fast rendering using Virtual DOM
- **Main Components:**
  - **Login/Register Component:** Handles user authentication input
  - **Chat Dashboard:** Displays active chat rooms and users
  - **Chat Window:** Shows real-time messages
  - **Message Input Box:** Allows users to type and send messages
- **Working:**
  - The frontend sends HTTP requests (using fetch/axios) to the backend APIs.
  - It also connects to the server using Socket.IO client for real-time communication.
  - State management is handled using React hooks like **useState** and **useEffect**.

5.2 Backend Implementation (Node.js + Express)

The backend is developed using **Node.js** and **Express.js**, which handle server-side logic and API development.

- **Key Responsibilities:**
  - Handle client requests
  - Manage authentication
  - Process chat messages
  - Interact with database

- **Structure:**

  - **Controllers:** Business logic (auth, messages, rooms)
  - **Routes:** API endpoints
  - **Models:** Database schemas
  - **Middleware:** Authentication & error handling

- **Database:**

  - Uses **MongoDB**
  - Stores users, messages, and chat rooms

5.3 Socket Implementation (Real-Time Messaging)

Real-time communication is implemented using **Socket.IO**.

- **Features:**

  - Bi-directional communication
  - Event-driven messaging
  - Instant message delivery

- **Workflow:**

  1. Client connects to server via socket
  2. User joins a chat room
  3. When a message is sent:

     - Event is emitted (sendMessage)
     - Server receives and processes it
     - Message stored in database
     - Server broadcasts (receiveMessage) to all users

  4. All connected users receive the message instantly

5.4 API Implementation

The backend provides REST APIs for communication between frontend and server.

1. Register API

Endpoint: `/register`
Method: POST

- **Request Body:**

```
{
  "name": "Mahesh",
  "email": "mahesh@gmail.com",
  "password": "123456"
}
```

- **Functionality:**

- Validates input data
- Encrypts password
- Stores user in database

2. Login API

Endpoint: /login
Method: POST

- **Request Body:**

```
{
  "email": "mahesh@gmail.com",
  "password": "123456"
}
```

- **Functionality:**

- Verifies credentials
- Generates JWT token
- Sends authentication response

3. Send Message API

Endpoint: /sendMessage
Method: POST

- **Request Body:**

```
{
  "senderId": "123",
  "message": "Hello"
}
```

- **Functionality:**

- Saves message to database
- Emits socket event for real-time delivery

4. Get Messages API

Endpoint: `/getMessages`
Method: GET

- **Functionality:**

- Fetches chat history
- Returns messages for a specific chat room

Chapter 6: Testing

Testing is a critical phase in the development of the **Real-Time Chat Application**. It ensures that the system functions correctly, meets user requirements, and performs efficiently under different conditions. Various testing techniques were applied to validate both functional and non-functional aspects of the system.

6.1 Objectives of Testing

- To verify that all features work as expected
- To identify and fix bugs or errors
- To ensure system reliability and stability
- To validate real-time communication functionality
- To check performance under multiple users

6.2 Types of Testing

1. Unit Testing

Unit testing involves testing individual components or modules of the system.

Examples:

- Testing login function
- Testing message sending function
- Testing database operations

Each module was tested independently to ensure correct functionality.

2. Integration Testing

Integration testing checks whether different modules work together correctly.

Examples:

- Frontend + Backend API communication
- Backend + Database interaction
- API + Socket.IO integration

Ensures smooth data flow across the system.

3. System Testing

System testing evaluates the complete application as a whole.

Examples:

- Full user flow (Register → Login → Chat → Logout)
- Chat room creation and usage
- Real-time message delivery

Confirms that the system meets requirements.

4. User Acceptance Testing (UAT)

Testing performed from the user's perspective.

Goals:

- Check usability
- Verify user experience
- Validate system behavior

Ensures the system is user-friendly and practical.

5. Performance Testing

Tests system behavior under load.

Parameters:

- Response time
- Message delivery speed
- Multiple users handling

The system showed low latency and smooth performance.

6. Security Testing

Ensures data protection and secure access.

Checks:

- Authentication validation
- Password encryption
- Unauthorized access prevention

6.3 Test Cases

Test Case ID	Description	Input	Expected Output	Result
TC01	User Registration	Valid details	Account created	Pass
TC02	User Login	Correct credentials	Login success	Pass
TC03	Invalid Login	Wrong password	Error message	Pass
TC04	Send Message	"Hello"	Message delivered	Pass
TC05	Receive Message	Incoming message	Display instantly	Pass
TC06	Chat Room Creation	Room name	Room created	Pass
TC07	Fetch Messages	Request history	Messages displayed	Pass

- **Postman:** API testing
- **Browser Developer Tools:** UI and debugging
- **MongoDB Compass:** Database validation

6.5 Error Handling

- Invalid login credentials handled properly
- Empty message validation implemented
- Server errors managed using middleware

6.6 Results of Testing

- All modules function correctly
- Real-time messaging works without delay
- System handles multiple users efficiently
- No critical bugs found after testing

6.7 Summary

The testing phase confirmed that the system is:

- Reliable
- Secure
- Efficient
- User-friendly

All functionalities of the **Real-Time Chat Application** were verified successfully, ensuring readiness for deployment and real-world usage.

Chapter 7: Results & Discussion

This chapter presents the outcomes of the developed **Real-Time Chat Application** and analyzes its performance, functionality, and overall effectiveness based on testing and implementation.

7.1 Results

The developed system was successfully implemented and tested under various conditions. The application achieved its primary objective of enabling real-time communication between users.

Key Results:

- **Real-Time Messaging:** Messages are delivered instantly without noticeable delay using Socket.IO.
- **User Authentication:** Secure login and registration functionality works correctly.
- **Chat Room Functionality:** Users can create and join chat rooms successfully.
- **Data Storage:** Messages and user data are stored and retrieved efficiently from the database.
- **User Interface:** The frontend is responsive, interactive, and easy to use.
- **Multi-user Support:** The system supports multiple users communicating simultaneously.

7.2 Performance Analysis

The application was evaluated based on performance metrics:

- **Response Time:** Fast response for API calls and message delivery
- **Latency:** Minimal delay in real-time communication
- **Scalability:** Able to handle multiple concurrent users
- **Stability:** System runs without crashes during testing

7.3 Discussion

The project demonstrates the effective use of full-stack development and real-time communication technologies. The integration of frontend, backend, database, and Socket.IO ensures smooth and efficient operation of the system.

Observations:

- Real-time communication significantly improves user experience
- Modular architecture makes the system easy to maintain and extend
- Use of modern technologies ensures scalability and performance

Limitations:

- Limited to text-based communication
- No media/file sharing functionality
- Basic security features (can be enhanced further)

7.4 Overall Evaluation

The system meets all the functional and non-functional requirements defined during the analysis phase. It provides a reliable and efficient platform for real-time communication.

7.5 Summary

The results confirm that the **Real-Time Chat Application** is:

- Technically sound
- Efficient in performance
- Scalable for future enhancements

The project successfully demonstrates practical implementation of real-time systems and serves as a strong foundation for further development.

Chapter 8: Conclusion & Future Scope

8.1 Conclusion

The **Real-Time Chat Application** developed in this project successfully demonstrates the implementation of a modern communication system using full-stack technologies. The application fulfills its primary objective of enabling instant messaging between users through a secure and efficient platform.

The system was designed and implemented using React.js for the frontend, Node.js and Express.js for the backend, MongoDB for data storage, and Socket.IO for real-time communication. All core functionalities such as user registration, login, chat room creation, and real-time message exchange were implemented successfully. The system performs efficiently with minimal latency and supports multiple users simultaneously.

This project helped in understanding key concepts such as REST API development, real-time event handling, database integration, and client-server architecture. It also provided practical exposure to problem-solving, debugging, and system design.

Overall, the project meets all functional and non-functional requirements and serves as a reliable and scalable communication platform.

8.2 Future Scope

Although the current system provides essential chat functionalities, there are several areas for enhancement and future development:

- **AI Chatbot Integration:** Implement intelligent chatbots for automated responses
- **Voice and Video Calling:** Add real-time audio and video communication features
- **File and Media Sharing:** Allow users to send images, documents, and videos
- **End-to-End Encryption:** Improve security for private communication
- **Push Notifications:** Notify users of new messages even when offline
- **Mobile Application:** Develop Android/iOS versions of the application
- **User Status Indicators:** Show online/offline and typing status

These enhancements can significantly improve the usability, scalability, and functionality of the system, making it comparable to modern communication platforms.

Chapter 9: References

1. React.js Documentation. (2024). Retrieved from https://react.dev/
2. Node.js Documentation. (2024). Retrieved from https://nodejs.org/
3. Express.js Documentation. (2024). Retrieved from https://expressjs.com/
4. MongoDB Documentation. (2024). Retrieved from https://www.mongodb.com/docs/
5. Socket.IO Documentation. (2024). Retrieved from https://socket.io/docs/
6. Postman Learning Center. (2024). Retrieved from https://learning.postman.com/
7. Visual Studio Code Documentation. (2024). Retrieved from https://code.visualstudio.com/docs
8. Mozilla Developer Network. (2024). JavaScript Guide. Retrieved from https://developer.mozilla.org/
9. W3Schools. (2024). Web Development Tutorials. Retrieved from https://www.w3schools.com/
10. Stack Overflow. (2024). Developer Community Discussions. Retrieved from https://stackoverflow.com/
11. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California.
12. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
13. Oracle. (2024). JavaScript Documentation. Retrieved from https://developer.oracle.com/
14. MongoDB Inc. (2024). MongoDB Atlas Documentation. Retrieved from https://www.mongodb.com/cloud/atlas
15. Krug, S. (2014). Don’t Make Me Think: A Common Sense Approach to Web Usability. New Riders.
16. Pressman, R. S. (2010). Software Engineering: A Practitioner’s Approach. McGraw-Hill.
17. Google Developers. (2024). Web Fundamentals. Retrieved from https://developers.google.com/
18. npm Inc. (2024). npm Documentation. Retrieved from https://docs.npmjs.com/
19. ECMAScript. (2024). JavaScript Language Specification. Retrieved from https://tc39.es/
20. TutorialsPoint. (2024). MERN Stack Tutorial. Retrieved from https://www.tutorialspoint.com/

Chapter 10: Appendices

10.1 User Manual

10.1.1 System Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or Higher
RAM	4 GB
Storage	20 GB Free Space
Internet	Broadband Connection

Software Requirements

Software	Version
Node.js	18.x or Higher
MongoDB	8.x
React.js	Latest
Express.js	Latest
Socket.IO	Latest
Visual Studio Code	Latest
Postman	Latest
Git	Latest
Web Browser	Google Chrome / Microsoft Edge / Firefox

10.1.2 Installation Steps

Step 1: Extract the Project

Extract the project folder and open it using **Visual Studio Code**.

Step 2: Install Backend Dependencies

Open the terminal inside the **server** folder and execute:

```
npm install
```

This command installs all required backend dependencies listed in the **package.json** file.

Step 3: Install Frontend Dependencies

Open another terminal inside the **client** folder and execute:

```
npm install
```

This installs all React packages required by the frontend application.

Step 4: Configure Environment Variables

Create a **.env** file inside the server folder and configure the following variables:

```
PORT=5000

MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_secret_key

JWT_EXPIRES_IN=7d
```

Step 5: Start MongoDB

Start the MongoDB server or connect the application to MongoDB Atlas using the configured connection string.

Step 6: Run the Backend Server

Execute the following command inside the **server** folder:

```
npm run dev
```

The backend server starts on the configured port (default: **5000**).

Step 7: Run the Frontend

Execute the following command inside the **client** folder:

```
npm start
```

The React application starts and opens automatically in the default web browser.

10.1.3 User Registration

1. Open the application.
2. Click the **Register** button.
3. Enter:
    - Full Name
    - Email Address
    - Password
4. Click **Create Account**.
5. The application validates the entered information.
6. If the details are valid, a new user account is created successfully.

10.1.4 User Login

1. Open the Login page.
2. Enter the registered email address.
3. Enter the password.
4. Click **Login**.
5. The system verifies the user credentials.
6. After successful authentication, a JSON Web Token (JWT) is generated and the user is redirected to the chat dashboard.

10.1.5 Dashboard

After login, the dashboard displays:

- Available users
- Existing chat rooms
- Group chat option
- Direct chat option
- Online/Offline user status
- Recent conversations

10.1.6 Creating a Group Chat

1. Click **Create Group**.
2. Enter the group name.
3. Select participants.
4. Click **Create**.
5. The group chat room is created successfully.

10.1.7 Starting a Direct Chat

1. Select any user from the user list.
2. Click **Start Chat**.
3. A private chat room is created automatically if it does not already exist.

10.1.8 Sending Messages

1. Select a chat room.
2. Type the message in the message input field.
3. Press **Enter** or click the **Send** button.
4. The message is stored in MongoDB and instantly delivered to all participants using Socket.IO.

10.1.9 Receiving Messages

Whenever another participant sends a message:

- The message is received instantly.
- No page refresh is required.
- Messages appear in chronological order.

10.1.10 Typing Indicator

When another user is typing:

- A **Typing...** indicator is displayed.
- The indicator automatically disappears once typing stops.

10.1.11 Online and Offline Status

The application automatically updates the user status.

- **Online** – User is currently connected.
- **Offline** – User has logged out or disconnected.
- **Last Seen** – Displays the last active time for offline users.

10.1.12 Message Management

Users can:

- View previous conversations.
- Retrieve chat history.
- Delete their own messages (Soft Delete).
- Continue conversations without losing previous messages.

10.1.13 Logout

1. Click the **Logout** button.
2. The application invalidates the current session.
3. The user's online status changes to **Offline**.
4. The user is redirected to the Login page.

10.1.14 Error Handling

The system displays appropriate error messages for invalid operations.

Examples include:

- Invalid email or password.
- Email already registered.
- Empty message not allowed.
- Unauthorized access.
- Room not found.
- Network or server errors.

10.2 Database Schema

The Real-Time Chat Application uses **MongoDB** as the database management system. Three collections are used to store application data: **User**, **ChatRoom**, and **Message**.

10.2.1 User Collection

The User collection stores user account information and authentication details.

Field Name	Data Type	Description
_id	ObjectId	Unique User ID
name	String	User Name
email	String	User Email
password	String	Encrypted Password
avatar	String	Profile Image
isOnline	Boolean	Online Status
lastSeen	Date	Last Active Time
socketId	String	Socket Connection ID
createdAt	Date	Account Creation Time
updatedAt	Date	Last Updated

Source Code (User Model)

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const userSchema = new mongoose.Schema({
    name: String,
    email: String,
    password: String,
    avatar: String,
    isOnline: Boolean,
    lastSeen: Date,
    socketId: String
},{
    timestamps:true
});
```

10.2.2 ChatRoom Collection

The ChatRoom collection stores group chats and direct conversations.

Field	Type	Description
_id	ObjectId	Room ID
name	String	Room Name
description	String	Room Description
roomType	String	Group or Direct
participants	ObjectId[]	Users in Room
admin	ObjectId	Room Creator
lastMessage	ObjectId	Latest Message
avatar	String	Room Image
createdAt	Date	Created Time
updatedAt	Date	Updated Time

Source Code (ChatRoom Model)

```
const chatRoomSchema = new mongoose.Schema({
    name:String,
    description:String,
    roomType:String,
    participants:[{
        type:mongoose.Schema.Types.ObjectId,
        ref:"User"
    }],
    admin:{
        type:mongoose.Schema.Types.ObjectId,
        ref:"User"
    },
    lastMessage:{
        type:mongoose.Schema.Types.ObjectId,
        ref:"Message"
    }
},{
    timestamps:true
});
```

10.2.3 Message Collection

The Message collection stores all chat messages.

Field	Type	Description
_id	ObjectId	Message ID
sender	ObjectId	Sender ID
roomId	ObjectId	Chat Room
content	String	Message
messageType	String	Text/Image/File
readBy	ObjectId[]	Read Receipts
isDeleted	Boolean	Soft Delete
createdAt	Date	Message Time

Source Code (Message Model)

```
const messageSchema = new mongoose.Schema({
    sender:{
        type:mongoose.Schema.Types.ObjectId,
        ref:"User"
    },
    content:String,
    roomId:{
        type:mongoose.Schema.Types.ObjectId,
        ref:"ChatRoom"
    },
    messageType:String,
    readBy:[{
        type:mongoose.Schema.Types.ObjectId,
        ref:"User"
    }]
},{
    timestamps:true
});
```

10.3 Source Code

10.3.1 MongoDB Connection (connectDB.js

```
const mongoose = require("mongoose");
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });

    console.log(`✅ MongoDB Connected: ${conn.connection.host}`);

    // Handle connection events
    mongoose.connection.on("disconnected", () => {
      console.warn("⚠️  MongoDB disconnected. Attempting to
reconnect...");
    });

    mongoose.connection.on("reconnected", () => {
      console.log("✅ MongoDB reconnected");
    });
  } catch (error) {
    console.error(`❌ MongoDB connection error:
${error.message}`);
    process.exit(1); // Exit process with failure
  }
};

module.exports = connectDB;
```

10.3.2 Backend Configuration (package.json)

```
{
  "name": "chat-app-server",
  "version": "1.0.0",
  "description": "Production-ready real-time chat application backend",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "jest --forceExit --detectOpenHandles --
testPathPattern=tests/"
  },
```

```
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.0.3",
    "socket.io": "^4.6.1"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "nodemon": "^3.0.2",
    "supertest": "^6.3.4"
  },
  "jest": {
    "testEnvironment": "node",
    "testTimeout": 30000
  },
  "directories": {
    "test": "tests"
  }
}
```

10.3.3 User Model (User.js)

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

/**
 * User Schema
 * Stores user credentials, profile info, and online status.
 * Password is automatically hashed before saving via pre-save hook.
 */
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Name is required"],
      trim: true,
      minlength: [2, "Name must be at least 2 characters"],
      maxlength: [50, "Name cannot exceed 50 characters"],
    },
```

```
    email: {
      type: String,
      required: [true, "Email is required"],
      unique: true,
      trim: true,
      lowercase: true,
      match: [/^\S+@\S+\.\S+$/, "Please provide a valid email address"],
    },
    password: {
      type: String,
      required: [true, "Password is required"],
      minlength: [6, "Password must be at least 6 characters"],
      select: false, // Never return password in queries by default
    },
    avatar: {
      type: String,
      default: "", // Could be a URL to a profile picture
    },
    isOnline: {
      type: Boolean,
      default: false,
    },
    lastSeen: {
      type: Date,
      default: Date.now,
    },
    socketId: {
      type: String,
      default: null,
    },
  },
  {
    timestamps: true, // Adds createdAt and updatedAt automatically
  }
);

// ─── Pre-Save Hook: Hash Password ──────────────────────────
userSchema.pre("save", async function (next) {
  // Only hash if password was modified (or is new)
  if (!this.isModified("password")) return next();

  try {
    const salt = await bcrypt.genSalt(12);
    this.password = await bcrypt.hash(this.password, salt);
```

```
    next();
  } catch (error) {
    next(error);
  }
});

// ─── Instance Method: Compare Password ──────────────────────
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

// ─── Instance Method: Get Public Profile ────────────────────
// Returns user data without sensitive fields
userSchema.methods.toPublicProfile = function () {
  return {
    _id: this._id,
    name: this.name,
    email: this.email,
    avatar: this.avatar,
    isOnline: this.isOnline,
    lastSeen: this.lastSeen,
    createdAt: this.createdAt,
  };
};

module.exports = mongoose.model("User", userSchema);
```

10.3.4 ChatRoom Model (ChatRoom.js)

```
const mongoose = require("mongoose");
const chatRoomSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      trim: true,
      maxlength: [100, "Room name cannot exceed 100 characters"],
    },
    description: {
      type: String,
      trim: true,
      maxlength: [300, "Description cannot exceed 300 characters"],
      default: "",
```

```
    },
    roomType: {
      type: String,
      enum: ["direct", "group"],
      default: "group",
    },
    participants: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
      },
    ],
    admin: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
    },
    lastMessage: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Message",
      default: null,
    },
    avatar: {
      type: String,
      default: "",
    },
  },
  {
    timestamps: true,
  }
);

// ── Validation: Direct rooms need exactly 2 participants
──────────────────────
chatRoomSchema.pre("save", function (next) {
  if (this.roomType === "direct" && this.participants.length !== 2) {
    return next(new Error("Direct rooms must have exactly 2
participants"));
  }
  if (this.roomType === "group" && !this.name) {
    return next(new Error("Group rooms must have a name"));
  }
  next();
});
```

```
// ── Index for participant-based queries
──────────────────────────────────────
chatRoomSchema.index({ participants: 1 });

module.exports = mongoose.model("ChatRoom", chatRoomSchema);
```

10.3.5 Message Model (Message.js)

```
const mongoose = require("mongoose");
const messageSchema = new mongoose.Schema(
  {
    sender: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: [true, "Sender is required"],
    },
    content: {
      type: String,
      required: [true, "Message content is required"],
      trim: true,
      maxlength: [2000, "Message cannot exceed 2000 characters"],
    },
    roomId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "ChatRoom",
      required: [true, "Room ID is required"],
    },
    messageType: {
      type: String,
      enum: ["text", "image", "file", "system"],
      default: "text",
    },
    readBy: [
      {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
      },
    ],
    isDeleted: {
      type: Boolean,
      default: false,
    },
  },
  {
```

```
    timestamps: true, // createdAt acts as the message timestamp
  }
);

messageSchema.index({ roomId: 1, createdAt: -1 });

module.exports = mongoose.model("Message", messageSchema);
```

10.3.6 Authentication Controller (authController.js)

```
const jwt = require("jsonwebtoken");
const User = require("../models/User");

// ── Helper: Sign JWT ──────────────────────────────────────────────
const signToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || "7d",
  });
};

// ── Helper: Create & Send Token Response ──────────────────────────
const sendTokenResponse = (user, statusCode, res) => {
  const token = signToken(user._id);

  res.status(statusCode).json({
    success: true,
    token,
    user: user.toPublicProfile(),
  });
};

/**
 * @desc    Register a new user
 * @route   POST /api/auth/register
 * @access  Public
 */
const register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;

    // Validate required fields
    if (!name || !email || !password) {
      return res.status(400).json({
        success: false,
        message: "Please provide name, email, and password.",
      });
    }

    // Check if email is already registered
    const existingUser = await User.findOne({ email: email.toLowerCase() });
    if (existingUser) {
      return res.status(409).json({
        success: false,
        message: "Email already registered. Please log in.",
      });
    }
```

```
    // Create user — password hashed automatically via pre-save hook
    const user = await User.create({ name, email, password });

    sendTokenResponse(user, 201, res);
  } catch (error) {
    next(error);
  }
};

/**
 * @desc    Login user
 * @route   POST /api/auth/login
 * @access  Public
 */
const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({
        success: false,
        message: "Please provide email and password.",
      });
    }

    // Explicitly select password field (excluded by default in schema)
    const user = await User.findOne({ email: email.toLowerCase() }).select(
      "+password"
    );

    if (!user || !(await user.comparePassword(password))) {
      return res.status(401).json({
        success: false,
        message: "Invalid email or password.",
      });
    }

    // Update online status on login
    user.isOnline = true;
    user.lastSeen = Date.now();
    await user.save({ validateBeforeSave: false });

    sendTokenResponse(user, 200, res);
  } catch (error) {
    next(error);
  }
};

/**
 * @desc    Get current logged-in user profile
 * @route   GET /api/auth/me
 * @access  Private
 */
const getMe = async (req, res, next) => {
  try {
    // req.user is already attached by protect middleware
    res.status(200).json({
      success: true,
      user: req.user.toPublicProfile(),
    });
```

```
  } catch (error) {
    next(error);
  }
};

/**
 * @desc    Logout user (set offline)
 * @route   POST /api/auth/logout
 * @access  Private
 */
const logout = async (req, res, next) => {
  try {
    await User.findByIdAndUpdate(req.user._id, {
      isOnline: false,
      lastSeen: Date.now(),
      socketId: null,
    });

    res.status(200).json({
      success: true,
      message: "Logged out successfully.",
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @desc    Get all users (for sidebar user list)
 * @route   GET /api/auth/users
 * @access  Private
 */
const getAllUsers = async (req, res, next) => {
  try {
    // Return all users except the current one
    const users = await User.find({ _id: { $ne: req.user._id } }).select(
      "name email avatar isOnline lastSeen"
    );

    res.status(200).json({
      success: true,
      count: users.length,
      users,
    });
  } catch (error) {
    next(error);
  }
};

module.exports = { register, login, getMe, logout, getAllUsers };
```

10.3.8 Message Controller (messageController.js)

```
const Message = require("../models/Message");
const ChatRoom = require("../models/ChatRoom");

/**
```

```
 * @desc    Get paginated messages for a room
 * @route   GET /api/messages/:roomId
 * @access  Private
 * @query   page (default 1), limit (default 50)
 */
const getMessages = async (req, res, next) => {
  try {
    const { roomId } = req.params;
    const page = parseInt(req.query.page) || 1;
    const limit = Math.min(parseInt(req.query.limit) || 50, 100); // Max
100 per page
    const skip = (page - 1) * limit;

    // Verify the user is a participant in this room
    const room = await ChatRoom.findOne({
      _id: roomId,
      participants: req.user._id,
    });

    if (!room) {
      return res.status(403).json({
        success: false,
        message: "Room not found or you are not a participant.",
      });
    }

    const [messages, total] = await Promise.all([
      Message.find({ roomId, isDeleted: false })
        .populate("sender", "name email avatar")
        .sort({ createdAt: -1 }) // Newest first (client reverses for
display)
        .skip(skip)
        .limit(limit),
      Message.countDocuments({ roomId, isDeleted: false }),
    ]);

    res.status(200).json({
      success: true,
      messages: messages.reverse(), // Return oldest-first for UI
rendering
      pagination: {
        total,
        page,
        limit,
        totalPages: Math.ceil(total / limit),
```

```
        hasMore: page * limit < total,
      },
    });
  } catch (error) {
    next(error);
  }
};

/**
 * @desc    Send a message to a room (REST fallback; Socket.IO is primary)
 * @route   POST /api/messages
 * @access  Private
 */
const sendMessage = async (req, res, next) => {
  try {
    const { roomId, content } = req.body;

    if (!roomId || !content?.trim()) {
      return res.status(400).json({
        success: false,
        message: "roomId and content are required.",
      });
    }

    // Ensure user is in the room
    const room = await ChatRoom.findOne({
      _id: roomId,
      participants: req.user._id,
    });

    if (!room) {
      return res.status(403).json({
        success: false,
        message: "Room not found or you are not a participant.",
      });
    }

    const message = await Message.create({
      sender: req.user._id,
      content: content.trim(),
      roomId,
    });

    // Update room's lastMessage pointer
```

```
    await ChatRoom.findByIdAndUpdate(roomId, { lastMessage: message._id
});

    const populated = await message.populate("sender", "name email
avatar");

    res.status(201).json({ success: true, message: populated });
  } catch (error) {
    next(error);
  }
};

/**
 * @desc    Soft-delete a message (only sender can delete)
 * @route   DELETE /api/messages/:id
 * @access  Private
 */
const deleteMessage = async (req, res, next) => {
  try {
    const message = await Message.findById(req.params.id);

    if (!message) {
      return res.status(404).json({ success: false, message: "Message not
found." });
    }

    if (message.sender.toString() !== req.user._id.toString()) {
      return res.status(403).json({
        success: false,
        message: "You can only delete your own messages.",
      });
    }

    message.isDeleted = true;
    message.content = "This message was deleted.";
    await message.save();

    res.status(200).json({ success: true, message: "Message deleted." });
  } catch (error) {
    next(error);
  }
};

module.exports = { getMessages, sendMessage, deleteMessage };
```

10.3.9 Socket.IO Handler (socketHandler.js)

```
const Message = require("../models/Message");
const ChatRoom = require("../models/ChatRoom");
const User = require("../models/User");

/**
 * Socket.IO Event Handler
 * Called once per authenticated socket connection.
 *
 * Events handled:
 *   ➜ joinRoom       – Join a chat room
 *   ➜ leaveRoom      – Leave a chat room
 *   ➜ sendMessage    – Broadcast a message to a room
 *   ➜ typing         – Broadcast typing indicator
 *   ➜ stopTyping     – Broadcast stop typing
 *   ➜ disconnect     – Mark user offline
 */
const socketHandler = (io) => {
  // Map to track userId → socketId for online presence
  const onlineUsers = new Map();

  io.on("connection", async (socket) => {
    const user = socket.user; // Attached by socketAuth middleware
    console.log(`🔌 Connected: ${user.name} (${socket.id})`);

    try {
      // ── Mark user as online
──────────────────────────────────────────────
      await User.findByIdAndUpdate(user._id, {
        isOnline: true,
        socketId: socket.id,
        lastSeen: Date.now(),
      });

      onlineUsers.set(user._id.toString(), socket.id);

      // Broadcast to ALL clients that this user is now online
      io.emit("userOnline", {
        userId: user._id,
        name: user.name,
        isOnline: true,
      });
```

```
    // ── Join Room ──────────────────────────────────────────────
    socket.on("joinRoom", async (roomId) => {
      try {
        // Verify membership before allowing join
        const room = await ChatRoom.findOne({
          _id: roomId,
          participants: user._id,
        });

        if (!room) {
          socket.emit("error", { message: "Room not found or access
denied." });
          return;
        }

        socket.join(roomId);
        console.log(`🟢 ${user.name} joined room: ${roomId}`);

        // Notify others in the room
        socket.to(roomId).emit("userJoined", {
          userId: user._id,
          name: user.name,
          roomId,
        });
      } catch (err) {
        console.error("joinRoom error:", err.message);
        socket.emit("error", { message: "Failed to join room." });
      }
    });

    // ── Leave Room ─────────────────────────────────────────────
    socket.on("leaveRoom", (roomId) => {
      socket.leave(roomId);
      console.log(`🔴 ${user.name} left room: ${roomId}`);

      socket.to(roomId).emit("userLeft", {
        userId: user._id,
        name: user.name,
        roomId,
      });
    });
```

```
      // ── Send Message ──────────────────────────────────────────
      socket.on("sendMessage", async (data) => {
        try {
          const { roomId, content } = data;

          if (!roomId || !content?.trim()) {
            socket.emit("error", { message: "roomId and content are
required." });
            return;
          }

          // Verify the user is still a participant
          const room = await ChatRoom.findOne({
            _id: roomId,
            participants: user._id,
          });

          if (!room) {
            socket.emit("error", { message: "Room not found or access
denied." });
            return;
          }

          // Persist message to MongoDB
          const message = await Message.create({
            sender: user._id,
            content: content.trim(),
            roomId,
          });

          // Update room's lastMessage
          await ChatRoom.findByIdAndUpdate(roomId, { lastMessage:
message._id });

          // Populate sender details before broadcasting
          const populated = await message.populate("sender", "name email
avatar");

          // Emit to ALL users in the room (including sender for
confirmation)
          io.to(roomId).emit("receiveMessage", populated);
        } catch (err) {
          console.error("sendMessage error:", err.message);
          socket.emit("error", { message: "Failed to send message." });
```

```
    }
  });

  // ── Typing Indicator ──────────────────────────────────────
  socket.on("typing", ({ roomId }) => {
    // Broadcast to room EXCEPT the sender
    socket.to(roomId).emit("userTyping", {
      userId: user._id,
      name: user.name,
      roomId,
    });
  });

  socket.on("stopTyping", ({ roomId }) => {
    socket.to(roomId).emit("userStopTyping", {
      userId: user._id,
      roomId,
    });
  });

  // ── Disconnect ────────────────────────────────────────────
  socket.on("disconnect", async () => {
    console.log(`🔴 Disconnected: ${user.name} (${socket.id})`);

    try {
      await User.findByIdAndUpdate(user._id, {
        isOnline: false,
        socketId: null,
        lastSeen: Date.now(),
      });

      onlineUsers.delete(user._id.toString());

      // Notify all clients that this user went offline
      io.emit("userOffline", {
        userId: user._id,
        name: user.name,
        isOnline: false,
        lastSeen: new Date(),
      });
    } catch (err) {
      console.error("disconnect handler error:", err.message);
    }
```

```
      });
    } catch (err) {
      console.error("Socket connection error:", err.message);
      socket.disconnect();
    }
  });
};

module.exports = socketHandler;
```

10.3.10 React Entry File (index.js)

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
reportWebVitals();
```

10.4 REST API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register User
POST	/api/auth/login	Login User
GET	/api/auth/me	Get User Profile
POST	/api/auth/logout	Logout User
GET	/api/auth/users	Retrieve Users
GET	/api/rooms	Retrieve Chat Rooms

Method	Endpoint	Description
POST	/api/rooms	Create Group Chat
POST	/api/rooms/direct	Create Direct Chat
GET	/api/rooms/:id	Get Room Details
DELETE	/api/rooms/:id	Delete Chat Room
GET	/api/messages/:roomId	Get Chat History
POST	/api/messages	Send Message
DELETE	/api/messages/:id	Delete Message

10.5 Project Structure

```
Chat-App
│
├── client
│   ├── public
│   ├── src
│   │   ├── components
│   │   ├── pages
│   │   ├── services
│   │   ├── hooks
│   │   ├── App.js
│   │   └── index.js
│
├── server
│   ├── config
│   ├── controllers
│   ├── middleware
│   ├── models
│   ├── routes
│   ├── socket
│   ├── server.js
│   └── package.json
│
└── README.md
```

10.6 Software Requirements

Software	Version
Node.js	Latest LTS
React.js	Latest
Express.js	Latest
MongoDB	Latest
Socket.IO	Latest
Visual Studio Code	Latest
Git	Latest
Postman	Latest

10.7 Hardware Requirements

Hardware	Specification
Processor	Intel Core i3 or Above
RAM	4 GB Minimum
Storage	20 GB Free Space
Operating System	Windows/Linux/macOS
Internet	Required

10.8 Screenshots

Figure 10.1 Registration Page

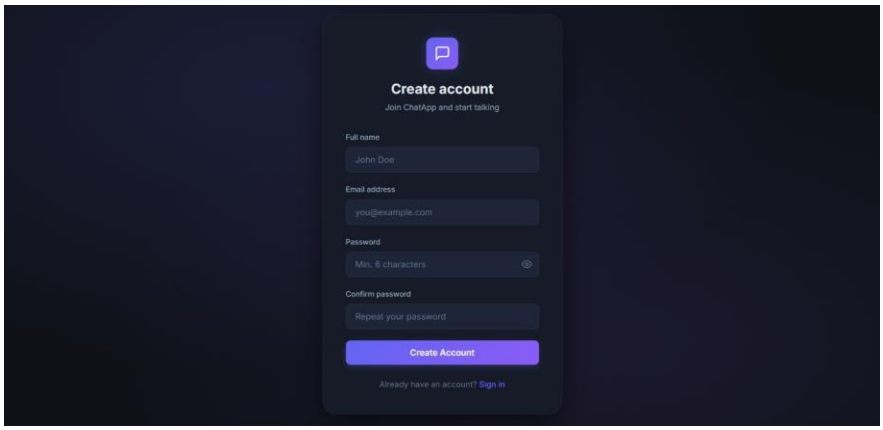
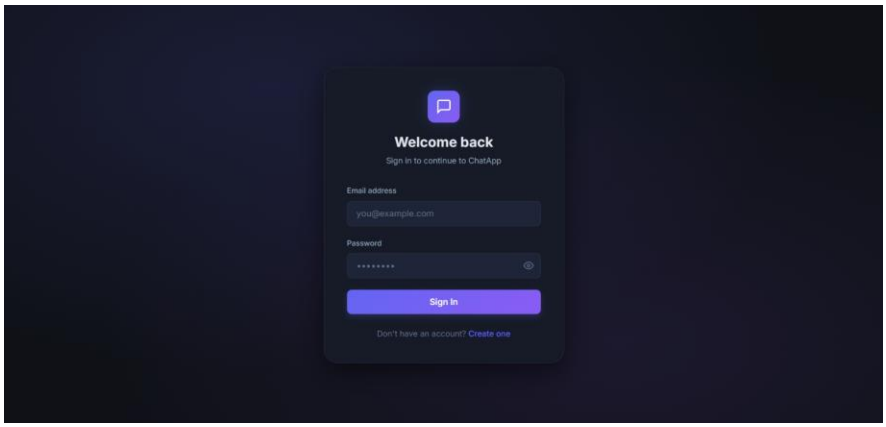
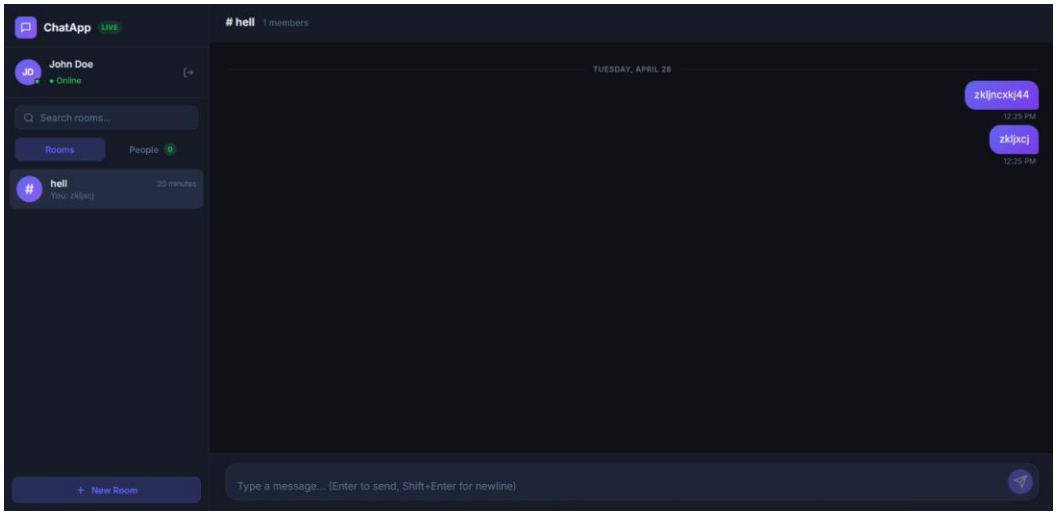


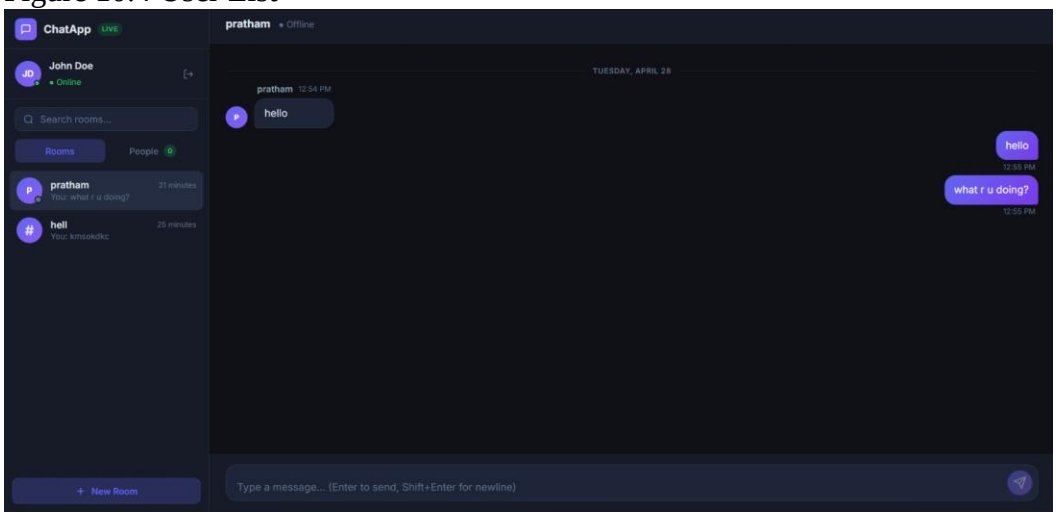
Figure 10.2 Login Page



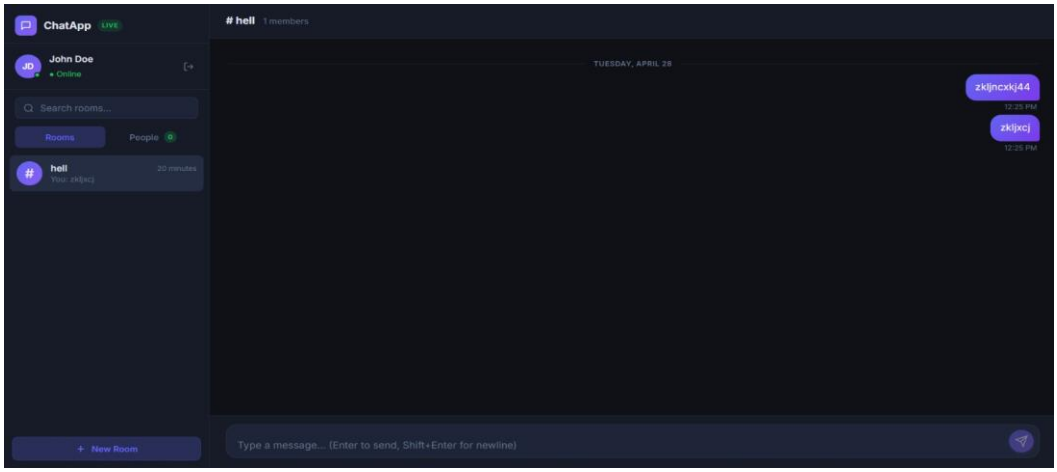
- Figure 10.3 Dashboard



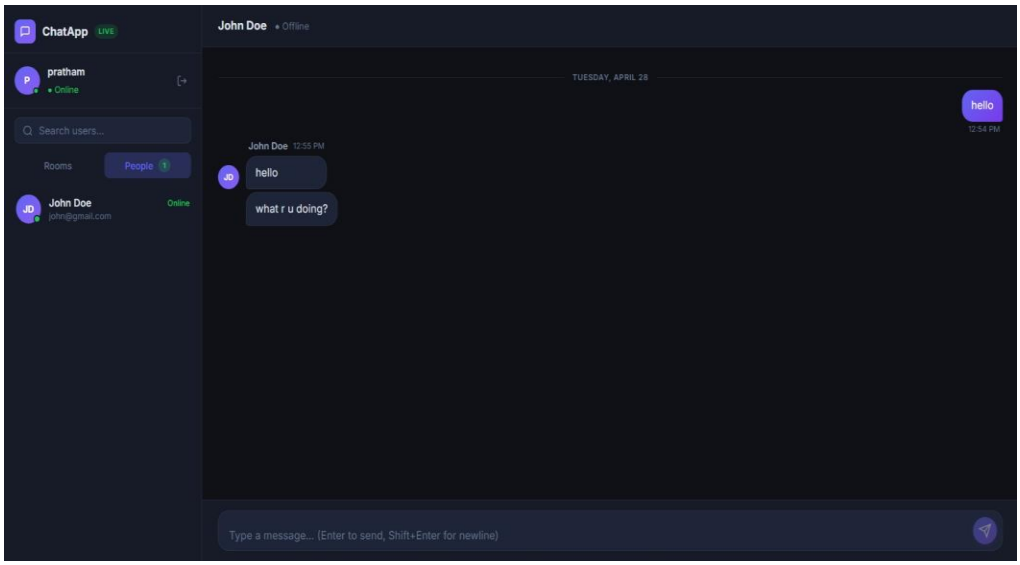
- Figure 10.4 User List



- Figure 10.5 Group Chat



- Figure 10.6 Direct Chat



10.9 Summary

The appendix presents additional technical information related to the Real-Time Chat Application, including the user manual, MongoDB database schema, important source code files, REST API endpoints, project structure, software and hardware requirements, and application screenshots. These supplementary materials provide a comprehensive understanding of the system architecture, implementation, and operation.